

American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Mid-Term Project Report

Summer 2025-2026

Section: D

Group: 09

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Eva Jahan ID: 23-55536-3	Tokenization, Case Folding, Lemmatization, Stop Word Removal, Punctuation Removal Logistic Regression — BoW & TF-IDF, Dataset 1 (Reuters) & Dataset 2 (Spooky Author) LR confusion matrices for Dataset 1 & Dataset 2
		Naive Bayes, and SVM — BoW & TF-IDF, Dataset 2 (Spooky Author) MNB and SVM confusion matrices for Dataset 2 Task 11: Results Comparison table + accuracy discussion Final packaging: comment-free .ipynb report assembly
2.	Name: Hamed Mohammed Adel ID: 23-55566-3	Extraction (BoW + TF-IDF vectorization, both datasets) Support Vector Machine — BoW & TF-IDF, Dataset 1 (Reuters) SVM confusion matrix for Dataset 1
		Data Loading (both datasets) Naive Bayes — BoW & TF-IDF, Dataset 1 (Reuters) NB confusion matrices for Dataset 1

Dataset Description

Dataset 1: Reuters50-50 (C50):

This dataset is from the subset of RCV1 and contains new articles written by the 50 top authors (respect to total size of news articles) with at least one subtopic of the class corporate/industrial were selected.

The training corpus consists of 2,500 texts (50 per author) and the test corpus includes other 2,500 texts (50 per author) which has no overlapping with the training corpus. Because the split is provided directly by the dataset, the existing train/test partition is used as is for this project.

Dataset 2: Spooky Author Identification:

This dataset, originally released as a Kaggle competition, contains horror fiction written by three classic authors: Edgar Allan Poe (EAP), HP Lovecraft (HPL), and Mary Shelley (MWS). The labeled train.csv has 19,579 rows which split into an 80% training set and a 20% test set.

Project Implementation Detail

Task 1: Data Loading

The first task was loads the raw dataset files into pandas DataFrames and keeps only the columns needed for classification, the article text and the author label. Rows with missing text or label are dropped. For Dataset 1, the Reuters 50-50 loaded directly and used as the train and test splits. But For dataset 2, the Spooky Author Identification train.csv is loaded and split 80/20 with stratification by author.

Code for task1:

```
df1 = pd.read_csv(REUTERS_TRAIN_DIR, encoding ='unicode_escape')
df2 = pd.read_csv(REUTERS_TEST_DIR, encoding ='unicode_escape')


train_data1 = df1[["text", "author"]]
train_data1.dropna(how='any', inplace=True)
print(train_data1.head())

test_data1 = df2[["text", "author"]]
test_data1.dropna(how='any', inplace=True)
print(test_data1.head())
```

```
df3 = pd.read_csv(Spooky_Aurthor_DIR)

train_data2, test_data2 = train_test_split(
    df3,
    test_size=0.2,
    random_state=42,
    stratify=df3["author"]
)

train_data2.dropna(how='any', inplace=True)
print(train_data2.head())

test_data2.dropna(how='any', inplace=True)
print(test_data2.head())
```

Output of Code for task1:

```
text  author
      0  The Internet continued to grow in leaps and bo... AaronPressman
      1  Elementary school students with access to the ... AaronPressman
      2  Hewlett-Packard Co. unveiled a new plan Monday... AaronPressman
      3  A U.S. District Court judge Wednesday barred n... AaronPressman
      4  With the Clinton administration busily reworki... AaronPressman
text  author
      0  U.S. lawmakers redrafting a legislative propos... AaronPressman
      1  On the Internet, where new products come and g... AaronPressman
      2  U.S. Senators on Tuesday sharply criticized a ... AaronPressman
      3  Senator Bob Kerrey is preparing legislation in... AaronPressman
      4  The largest online services providers reacted ... AaronPressman
```

```
id                                                text author
4330   id23740  I never sought to return to those tenebrous la...    HPL
18391  id16593  Among them the Greek army was encamped, and th...    MWS
12928  id09546  claws and teeth sharpened on centuries of corp...    HPL
4925   id21267  Goaded, by the interference, into a rage more ...    EAP
17281  id20831  The whole company, indeed, seemed now deeply t...    EAP
            id                                                text author
19375  id12024  So blended bank and shadow there That each see...    EAP
15714  id15178  I did only a retail business myself, but my ol...    EAP
15451  id18559  Old Waldron, who had curtailed his activities ...    HPL
7280   id21414  Let us refer, for example, to the last instanc...    EAP
```

Code Description of task 1:

Here pd.read_csv reads each CSV into a DataFrame. Only the text and author columns are retained and dropna removes any rows that have missing data (author/text). For dataset 2, `train_test_split(df3,test_size=0.2,random_state=42,stratify=df3["author"])` is used to split dataset into test data 20% with random state 42 that suffle data before splitting and stratify keep the same class proportions in both datasets.

Task 2: Tokenization

Each raw text is split into individual word and punctuation tokens using NLTK word_tokenize.

Code for task 2:

```
train_data1["tokens"] = train_data1["text"].apply(word_tokenize)
print(train_data1[["text", "tokens"]].head())
test_data1["tokens"] = test_data1["text"].apply(word_tokenize)
print(test_data1[["text", "tokens"]].head())
```

```
train_data2["tokens"] = train_data2["text"].apply(word_tokenize)
print(train_data2[["text", "tokens"]].head())
test_data2["tokens"] = test_data2["text"].apply(word_tokenize)
print(test_data2[["text", "tokens"]].head())
```

Output of Code for task 2:

```
                                                text  \
0  The Internet continued to grow in leaps and bo...
1  Elementary school students with access to the ...
2  Hewlett-Packard Co. unveiled a new plan Monday...
3  A U.S. District Court judge Wednesday barred n...
4  With the Clinton administration busily reworki...
                                              tokens
0  [The, Internet, continued, to, grow, in, leaps...
1  [Elementary, school, students, with, access, t...
2  [Hewlett-Packard, Co., unveiled, a, new, plan,...
3  [A, U.S., District, Court, judge, Wednesday, b...
4  [With, the, Clinton, administration, busily, r...
                                                text  \
0  U.S. lawmakers redrafting a legislative propos...
1  On the Internet, where new products come and g...
2  U.S. Senators on Tuesday sharply criticized a ...
3  Senator Bob Kerrey is preparing legislation in...
4  The largest online services providers reacted ...
                                              tokens
0  [U.S., lawmakers, redrafting, a, legislative, ...
1  [On, the, Internet, ,, where, new, products, c...
2  [U.S, ., Senators, on, Tuesday, sharply, criti...
3  [Senator, Bob, Kerrey, is, preparing, legislat...
```

```
id                                                  text author
4330   id23740  I never sought to return to those tenebrous la...    HPL
18391  id16593  Among them the Greek army was encamped, and th...    MWS
12928  id09546  claws and teeth sharpened on centuries of corp...    HPL
4925   id21267  Goaded, by the interference, into a rage more ...    EAP
17281  id20831  The whole company, indeed, seemed now deeply t...    EAP
            id                                                text author
19375  id12024  So blended bank and shadow there That each see...    EAP
15714  id15178  I did only a retail business myself, but my ol...    EAP
15451  id18559  Old Waldron, who had curtailed his activities ...    HPL
7280   id21414  Let us refer, for example, to the last instanc...    EAP
6051   id25439  The first intelligence of the murder was broug...    EAP
```

Code Description of task 2:

Here word_tokenize is applied to the text column using apply() which generate a new tokens column where each row holds a list of tokens instead of a string.

Task 3 : Case Folding

Every token is converted into lowercase so that the same word appearing with different capitalization is treated as single vocabulary entry rather than two different features.

Code for task 3:

```
train_data1["tokens"] = [
    [word.lower() for word in words]
    for words in train_data1["tokens"]
]
print(train_data1["tokens"].head())

test_data1["tokens"] = [
    [word.lower() for word in words]
    for words in test_data1["tokens"]
]
print(test data1["tokens"].head())
```

```
train_data2["tokens"] = [
    [word_tokenized.lower() for word_tokenized in words]
    for words in train_data2["tokens"]
]
print(train_data2["tokens"].head())
test_data2["tokens"] = [
    [word_tokenized.lower() for word_tokenized in words]
    for words in test_data2["tokens"]
]
print(test_data2["tokens"].head())
```

Output of Code for task 3:

```
0    [the, internet, continued, to, grow, in, leaps...
1    [elementary, school, students, with, access, t...
2    [hewlett-packard, co., unveiled, a, new, plan,...
3    [a, u.s., district, court, judge, wednesday, b...
4    [with, the, clinton, administration, busily, r...
Name: tokens, dtype: object
0    [u.s., lawmakers, redrafting, a, legislative, ...
1    [on, the, internet, ,, where, new, products, c...
2    [u.s, ., senators, on, tuesday, sharply, criti...
3    [senator, bob, kerrey, is, preparing, legislat...
4    [the, largest, online, services, providers, re...
```

```
4330     [i, never, sought, to, return, to, those, tene...
18391    [among, them, the, greek, army, was, encamped,...
12928    [claws, and, teeth, sharpened, on, centuries, ...
4925     [goaded, ,, by, the, interference, ,, into, a,...
17281    [the, whole, company, ,, indeed, ,, seemed, no...
Name: tokens, dtype: object
19375    [so, blended, bank, and, shadow, there, that, ...
15714    [i, did, only, a, retail, business, myself, ,,...
15451    [old, waldron, ,, who, had, curtailed, his, ac...
7280     [let, us, refer, ,, for, example, ,, to, the, ...
6051     [the, first, intelligence, of, the, murder, wa...
```

Code Description of task 3:

A nested list comprehension iterates over every token list and lowercases each token with .lower().

Task 4: Lemmatization

Every token is reduced to its dictionary base form (lemma) using spaCy's en_core_web_sm model, so inflected forms like "continued" or "unveiled" are normalized to "continue" and "unveil". This reduces vocabulary sparsity and groups together words that differ only by tense or number.

Code for task 4:

```
nlp = spacy.load("en_core_web_sm")
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)
print(train_data1["tokens"].head())

test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)
print(test_data1["tokens"].head())
```

```
train_data2["tokens"] = train_data2["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)
print(train_data2["tokens"].head())


test_data2["tokens"] = test_data2["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)
print(test_data2["tokens"].head())
```

Output of Code for task 4:

```
0    [the, internet, continue, to, grow, in, leap, ...
1    [elementary, school, student, with, access, to...
2    [hewlett, -, packard, co., unveil, a, new, pla...
3    [a, u.s, ., district, court, judge, wednesday,...
4    [with, the, clinton, administration, busily, r...
Name: tokens, dtype: object
0    [u.s, ., lawmaker, redrafte, a, legislative, p...
1    [on, the, internet, ,, where, new, product, co...
2    [u.s, ., senator, on, tuesday, sharply, critic...
3    [senator, bob, kerrey, be, prepare, legislatio...
4    [the, large, online, service, provider, react,...
```

```
4330     [I, never, seek, to, return, to, those, tenebr...
18391    [among, they, the, greek, army, be, encamp, ,,...
12928    [claws, and, tooth, sharpen, on, century, of, ...
4925     [goad, ,, by, the, interference, ,, into, a, r...
17281    [the, whole, company, ,, indeed, ,, seem, now,...
Name: tokens, dtype: object
19375    [so, blend, bank, and, shadow, there, that, ea...
15714    [I, do, only, a, retail, business, myself, ,, ...
15451    [old, waldron, ,, who, have, curtail, his, act...
7280     [let, we, refer, ,, for, example, ,, to, the, ...
6051     [the, first, intelligence, of, the, murder, be...
```

Code Description of task 4:

spaCy lemmatizer needs “doc” object rather than a raw token list. So the lowercased tokens are rejoined into a string with .join(words), and processed through nlp pipeline, and resulting spaCy token;s .lemma_ attribute is collected back into a list.

Task 5: Stop Word Removal

Common English word that have very little meaning or do not have significant meaning on their own such as “a”, “an”, “the”. Using NLTK’s built-in English stop word list.

Code for task 5:

```
stop_words = set(stopwords.words("english"))
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)
print(train_data1["tokens"].head())

test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)
print(test_data1["tokens"].head())
```

```
train_data2["tokens"] = train_data2["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)
print(train_data2["tokens"].head())

test_data2["tokens"] = test_data2["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)
print(test_data1["tokens"].head())
```

Output of Code for task 5:

```
0    [internet, continue, grow, leap, bound, year, ...
1    [elementary, school, student, access, internet...
2    [hewlett, -, packard, co., unveil, new, plan, ...
3    [u.s, ., district, court, judge, wednesday, ba...
4    [clinton, administration, busily, rework, 1995...
Name: tokens, dtype: object
0    [u.s, ., lawmaker, redrafte, legislative, prop...
1    [internet, ,, new, product, come, go, blink, e...
2    [u.s, ., senator, tuesday, sharply, criticize,...
3    [senator, bob, kerrey, prepare, legislation, a...
4    [large, online, service, provider, react, caut...
```

```
4330     [I, never, seek, return, tenebrous, labyrinth,...
18391    [among, greek, army, encamp, ,, squadron, move...
12928     [claws, tooth, sharpen, century, corps, ., ., .]
4925     [goad, ,, interference, ,, rage, demoniacal, ,...
17281    [whole, company, ,, indeed, ,, seem, deeply, f...
Name: tokens, dtype: object
0    [u.s, lawmaker, redrafte, legislative, proposa...
1    [internet, new, product, come, go, blink, eye,...
2    [u.s, senator, tuesday, sharply, criticize, ne...
3    [senator, bob, kerrey, prepare, legislation, a...
4    [large, online, service, provider, react, caut...
```

Code Description of task 5:

This code removes English stop words from the tokenized text using the NLTK stop word corpus. First, a set of English stop words is created using stopwords.words("english"). Then, the apply () function processes each list of tokens in the training and testing datasets.

Task 6: Punctuation Removal

This is the process of removing special character and punctuation marks from the text. This carries no significant meaning in authorship identification for a bag of word or tf-idf.

Code for task 6:

```
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [word for word in words if word not in
string.punctuation]
)
print(train_data1["tokens"].head())
test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [word for word in words if word not in
string.punctuation]
)
print(test_data1["tokens"].head())
```

```
train_data2["tokens"] = train_data1["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)
print(train_data2["tokens"].head())
test_data2["tokens"] = test_data1["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)
print(test_data2["tokens"].head())
```

Output of Code for task 6:

```
0    [internet, continue, grow, leap, bound, year, ...
1    [elementary, school, student, access, internet...
2    [hewlett, packard, co., unveil, new, plan, mon...
3    [u.s, district, court, judge, wednesday, bar, ...
4    [clinton, administration, busily, rework, 1995...
Name: tokens, dtype: object
0    [u.s, lawmaker, redrafte, legislative, proposa...
1    [internet, new, product, come, go, blink, eye,...
2    [u.s, senator, tuesday, sharply, criticize, ne...
3    [senator, bob, kerrey, prepare, legislation, a...
4    [large, online, service, provider, react, caut...
```

```
330     [I, never, seek, return, tenebrous, labyrinth,...
18391    [among, greek, army, encamp, squadron, move, f...
12928              [claws, tooth, sharpen, century, corps]
4925     [goad, interference, rage, demoniacal, I, with...
17281    [whole, company, indeed, seem, deeply, feel, c...
Name: tokens, dtype: object
19375    [blend, bank, shadow, seem, pendulous, air, mi...
15714    [I, retail, business, old, habit, system, carr...
15451    [old, waldron, curtail, activity, would, make,...
7280     [let, refer, example, last, instance, one, com...
6051       [first, intelligence, murder, bring, g, person]
```

Code Description of task 6:

This code removes punctuation marks from the tokenized text in both the training and testing datasets. The apply () function processes each list of tokens, while the lambda function iterates through every word and keeps only those that are not present in string.punctuation

Task 7: Vector Semantics (Bag of Words)

Two vector representations are built from the processed text: a Bag of Words (BoW) representation using CountVectorizer, which represents each document as raw word counts.

Code for task 7:

```
bow = CountVectorizer()
X_train_bow = bow.fit_transform(train_data1["processed_text"])
X_test_bow = bow.transform(test_data1["processed_text"])
y_train = train_data1["author"]
y_test = test_data1["author"]
print(X_train_bow.shape)
print(X_test_bow.shape)
```

```
bow = CountVectorizer()
X_train2_bow = bow.fit_transform(train_data2["processed_text"])
X_test2_bow = bow.transform(test_data2["processed_text"])
y_train2 = train_data2["author"]
y_test2 = test_data2["author"]
print(X_train2_bow.shape)
print(X_test2_bow.shape)
```

Output of Code for task 7:

```
(2500, 23345)
(2500, 23345)
```

```
(15663, 17490)
(3916, 17490)
```

Code Description of task 7:

This code converts the pre-processed text into a numerical representation using the **Bag of Words (BoW)** model. The CountVectorizer() object creates a vocabulary of all unique words from the training dataset and represents each document by the frequency of those words. The fit_transform() method is applied to the training data to learn the vocabulary and generate the document-term matrix, while the transform () method is applied to the testing data using the same vocabulary. The class labels are stored in y_train and y_test, and the shape of the generated matrices is displayed to verify the number of documents and extracted features.

Task 8: Vector Semantics (TF-IDF)

TF-IDF representation using TfidfVectorizer, which re-weights word counts by how distinctive a word is across the corpus

Code for task 8:

```
tfidf = TfidfVectorizer()
X_train_tfidf = tfidf.fit_transform(train_data1["processed_text"])
X_test_tfidf = tfidf.transform(test_data1["processed_text"])
print(X_train_tfidf.shape)
print(X_test_tfidf.shape)
```

```
tfidf = TfidfVectorizer()
X_train2_tfidf = tfidf.fit_transform(train_data2["processed_text"])
X_test2_tfidf = tfidf.transform(test_data2["processed_text"])
print(X_train2_tfidf.shape)
print(X_test2_tfidf.shape)
```

Output of Code for task 8:

```
(2500, 23345)
(2500, 23345)
```

```
(15663, 17490)
(3916, 17490)
```

Code Description of task 8:

This code converts the preprocessed text into numerical feature vectors using the **TF-IDF** representation. The TfidfVectorizer() calculates the importance of each word based on its frequency within a document and its rarity across all documents. The fit_transform() method is used on the training dataset to learn the vocabulary and compute TF-IDF weights, while the transform () method applies the same vocabulary and weighting scheme to the testing dataset. Finally, the dimensions of the resulting TF-IDF matrices are displayed to confirm the number of documents and extracted features.

Task 9: Naïve Bayes Classification

Naïve Bayes is a probabilistic classifier. MultinomialNB, the standard Naïve Bayes variant for word-count style features.

Code (Bag of Words):

```
mnb_bow = MultinomialNB()

mnb_bow.fit(X_train_bow, y_train)

y_pred_mnb_bow = mnb_bow.predict(X_test_bow)


print("Naive Bayes-Bag of words")

print("Accuracy :", accuracy_score(y_test, y_pred_mnb_bow))

print("Precision:", precision_score(y_test, y_pred_mnb_bow,
average="weighted"))

print("Recall   :", recall_score(y_test, y_pred_mnb_bow,
average="weighted"))

print("F1 Score :", f1_score(y_test, y_pred_mnb_bow, average="weighted"))


mnb_bow_cm = confusion_matrix(y_test, y_pred_mnb_bow)

fig, ax = plt.subplots(figsize=(25, 25))


ConfusionMatrixDisplay(

    confusion_matrix=mnb_bow_cm,

    display_labels=mnb_bow.classes_

).plot(ax=ax, cmap="Greens", xticks_rotation=90)


plt.title("Multinomial Naive Bayes - Bag of Words")

plt.show()
```

Output Naïve Bayes Classification(Bag of words):

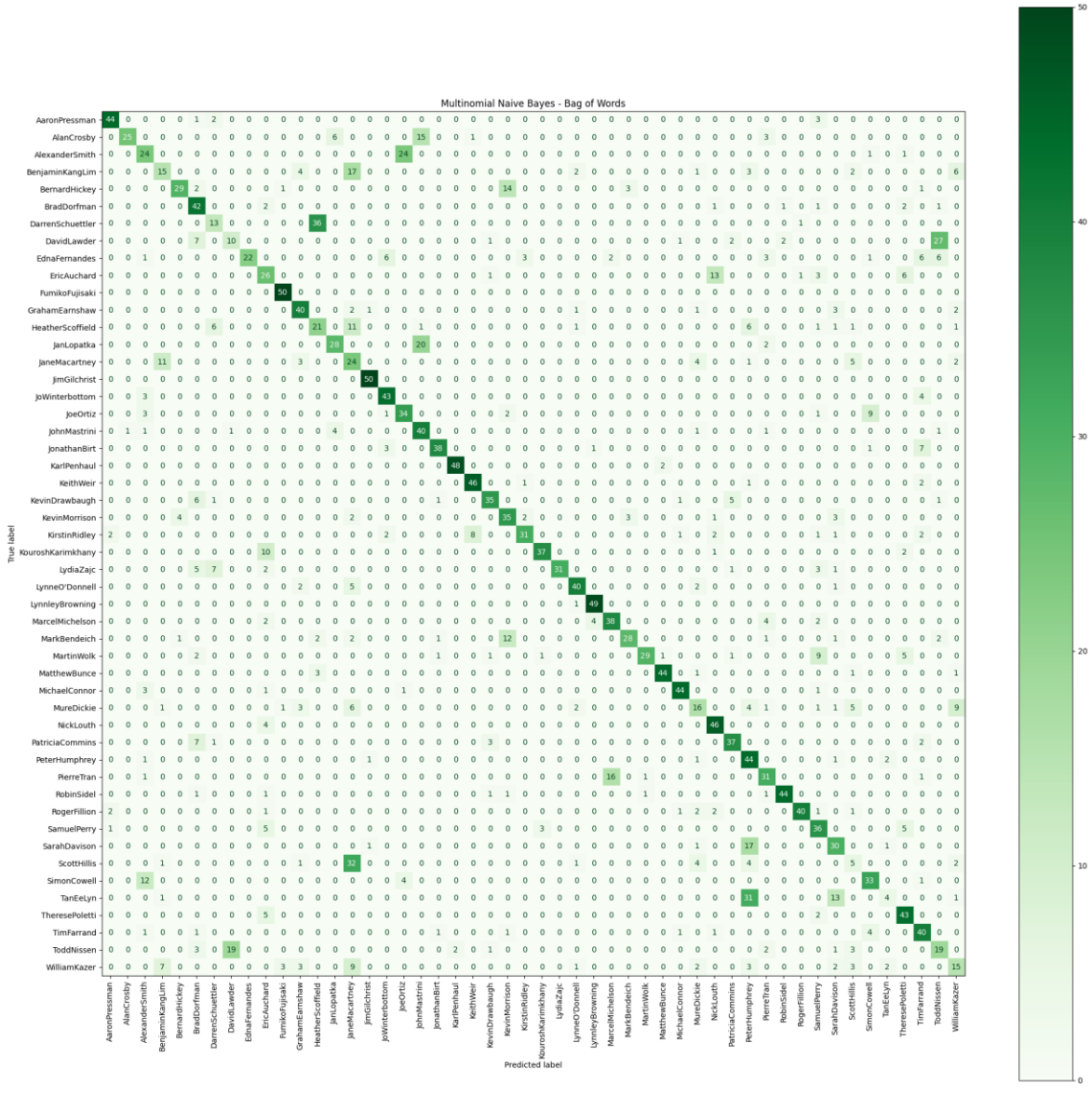
```
Naïve Bayes-Bag of words

Accuracy : 0.6544

Precision: 0.6814089651229854

Recall   : 0.6544

F1 Score : 0.6502480688741096
```



Code Description of Naïve Bayes Classification (BoW):

This code trains and evaluates a Multinomial Naïve Bayes (MultinomialNB) classifier using the Bag of Words (BoW) feature representation. First, a MultinomialNB model is created and trained using the BoW training data (X_train_bow) and the corresponding author labels (y_train) through the fit() method. After training, the predict() method is used to classify the testing documents and generate the predicted author labels (y_pred_mnb_bow). The model's

performance is then evaluated using four classification metrics: Accuracy, Precision, Recall, and F1-score, providing a comprehensive assessment of its classification performance. Finally, a confusion matrix is generated using confusion_matrix() and visualized with ConfusionMatrixDisplay. The confusion matrix illustrates the number of correct and incorrect predictions for each author class, making it easier to analyze the classification performance and identify any misclassified documents.

Code (TF-IDF):

```
mnb_tfidf = MultinomialNB()

mnb_tfidf.fit(X_train_tfidf, y_train)

y_pred_mnb_tfidf = mnb_tfidf.predict(X_test_tfidf)


print("Naive Bayes - TF-IDF")

print("Accuracy :", accuracy_score(y_test, y_pred_mnb_tfidf))

print("Precision:", precision_score(y_test, y_pred_mnb_tfidf,
average="weighted"))

print("Recall   :", recall_score(y_test, y_pred_mnb_tfidf,
average="weighted"))

print("F1 Score :", f1_score(y_test, y_pred_mnb_tfidf,
average="weighted"))


mnb_tfidf_cm = confusion_matrix(y_test, y_pred_mnb_tfidf)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(

    confusion_matrix=mnb_tfidf_cm,

    display_labels=mnb_tfidf.classes_

).plot(ax=ax, cmap="Blues", xticks_rotation=90)

plt.title("Multinomial Naive Bayes - TF-IDF")

plt.show()
```

Output Naïve Bayes Classification(TF-IDF):

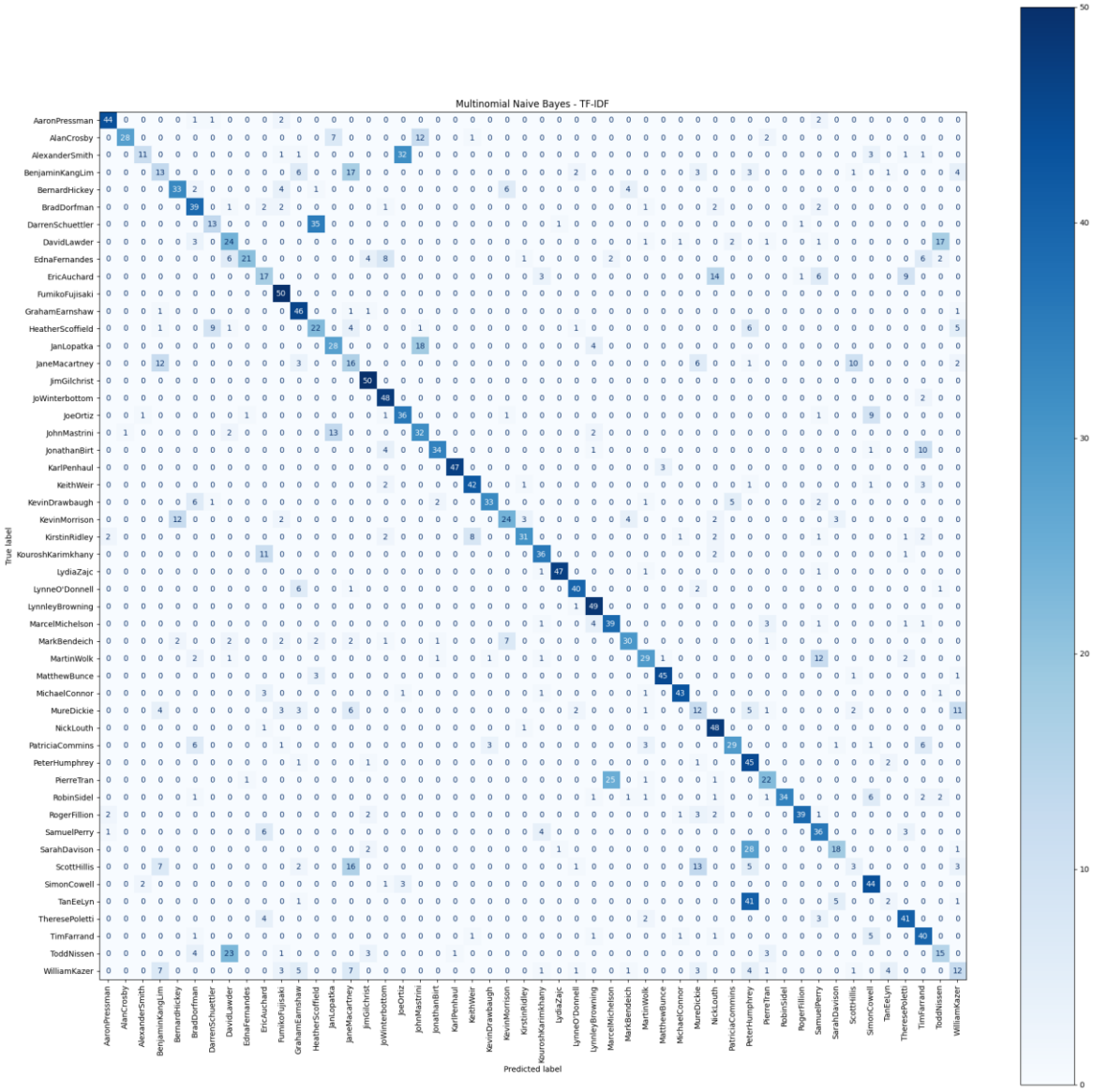
```
Naïve Bayes-Bag of words

Accuracy : 0.6544

Precision: 0.6814089651229854

Recall   : 0.6544

F1 Score : 0.6502480688741096
```



Code Description of Naïve Bayes Classification (TF-IDF):

This code trains and evaluates a **Multinomial Naïve Bayes (MultinomialNB)** classifier using the **TF-IDF** feature representation. First, a MultinomialNB model is created and trained with the TF-IDF training feature matrix (X_train_tfidf) and the corresponding author labels (y_train) using the fit() method. After the training process, the predict() method is used to classify the testing documents and generate the predicted author labels (y_pred_mnb_tfidf). The model's

performance is then evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-score** to measure its effectiveness in identifying the correct authors. Finally, a **confusion matrix** is generated and displayed using ConfusionMatrixDisplay, providing a visual representation of the model's correct and incorrect predictions for each author class.

Task 10: Logistic Regression

Logistic Regression is trained and evaluated the same way as Naive Bayes, on both the BoW and TF-IDF.

Code (Bag of Words):

```
lr_bow = LogisticRegression(max_iter=1000)
lr_bow.fit(X_train_bow, y_train)
y_pred_lr_bow = lr_bow.predict(X_test_bow)


print("Logistic Regression - Bag of words")
print("Accuracy :", accuracy_score(y_test, y_pred_lr_bow))
print("Precision:", precision_score(y_test, y_pred_lr_bow,
average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_bow,
average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_bow, average="weighted"))


lr_bow_cm = confusion_matrix(y_test, y_pred_lr_bow)
fig, ax = plt.subplots(figsize=(25, 25))
ConfusionMatrixDisplay(
    confusion_matrix=lr_bow_cm,
    display_labels=lr_bow.classes_
).plot(ax=ax, cmap="Reds", xticks_rotation=90)
plt.title("Logistic Regression-BoW")
plt.show()
plt.show()
```

Output **Logistic Regression** (Bag of words):

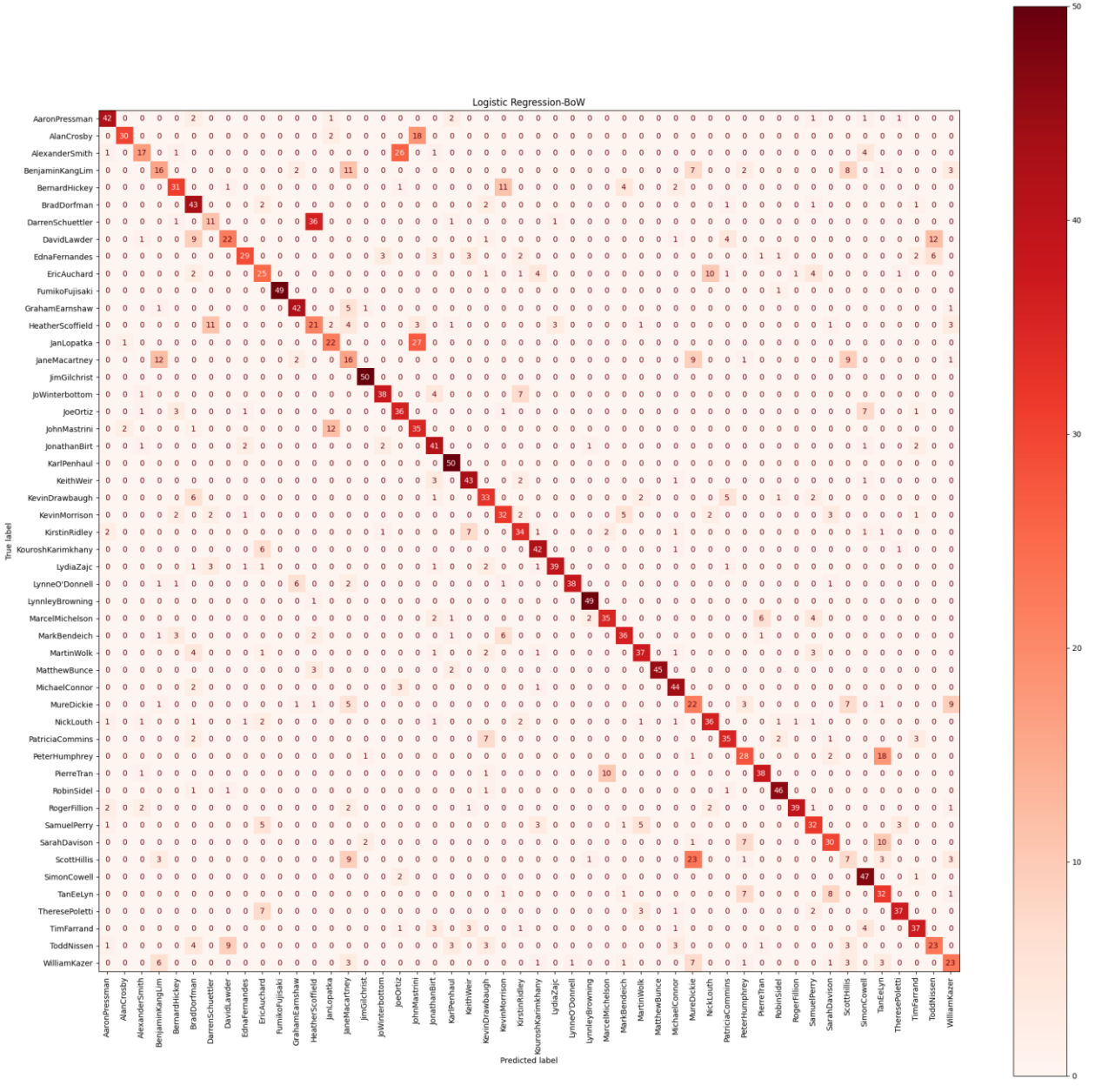
```
Logistic Regression - Bag of words

Accuracy : 0.674

Precision: 0.6855635801422039

Recall   : 0.674

F1 Score : 0.6720250979452913
```



Code Description of Logistic Regression (BoW):

This code trains and evaluates a **Logistic Regression** classifier using the **Bag of Words (BoW)** feature representation. First, a LogisticRegression model is created with max_iter=1000 to ensure sufficient iterations for model convergence during training. The model is then trained

using the Bag of Words training feature matrix (X_train_bow) and the corresponding author labels (y_train) through the fit() method. After training, the predict() method is used to classify the testing documents and generate the predicted author labels (y_pred_lr_bow). The model's performance is evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-score**. Finally, a **confusion matrix** is generated and displayed using ConfusionMatrixDisplay to visualize the correct and incorrect predictions for each author class.

Code (TF-IDF):

```
lr_tfidf = LogisticRegression(max_iter=1000)
lr_tfidf.fit(X_train_tfidf, y_train)
y_pred_lr_tfidf = lr_tfidf.predict(X_test_tfidf)


print("Logistic Regression - TF-IDF")
print("Accuracy :", accuracy_score(y_test, y_pred_lr_tfidf))
print("Precision:", precision_score(y_test, y_pred_lr_tfidf,
average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_tfidf,
average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_tfidf, average="weighted"))


lr_tfidf_cm = confusion_matrix(y_test, y_pred_lr_tfidf)
fig, ax = plt.subplots(figsize=(25, 25))
ConfusionMatrixDisplay(
    confusion_matrix=lr_tfidf_cm,
    display_labels=lr_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)
plt.title("Logistic Regression- TF-IDF")
plt.show()
```

Output Logistic Regression (TF-IDF):

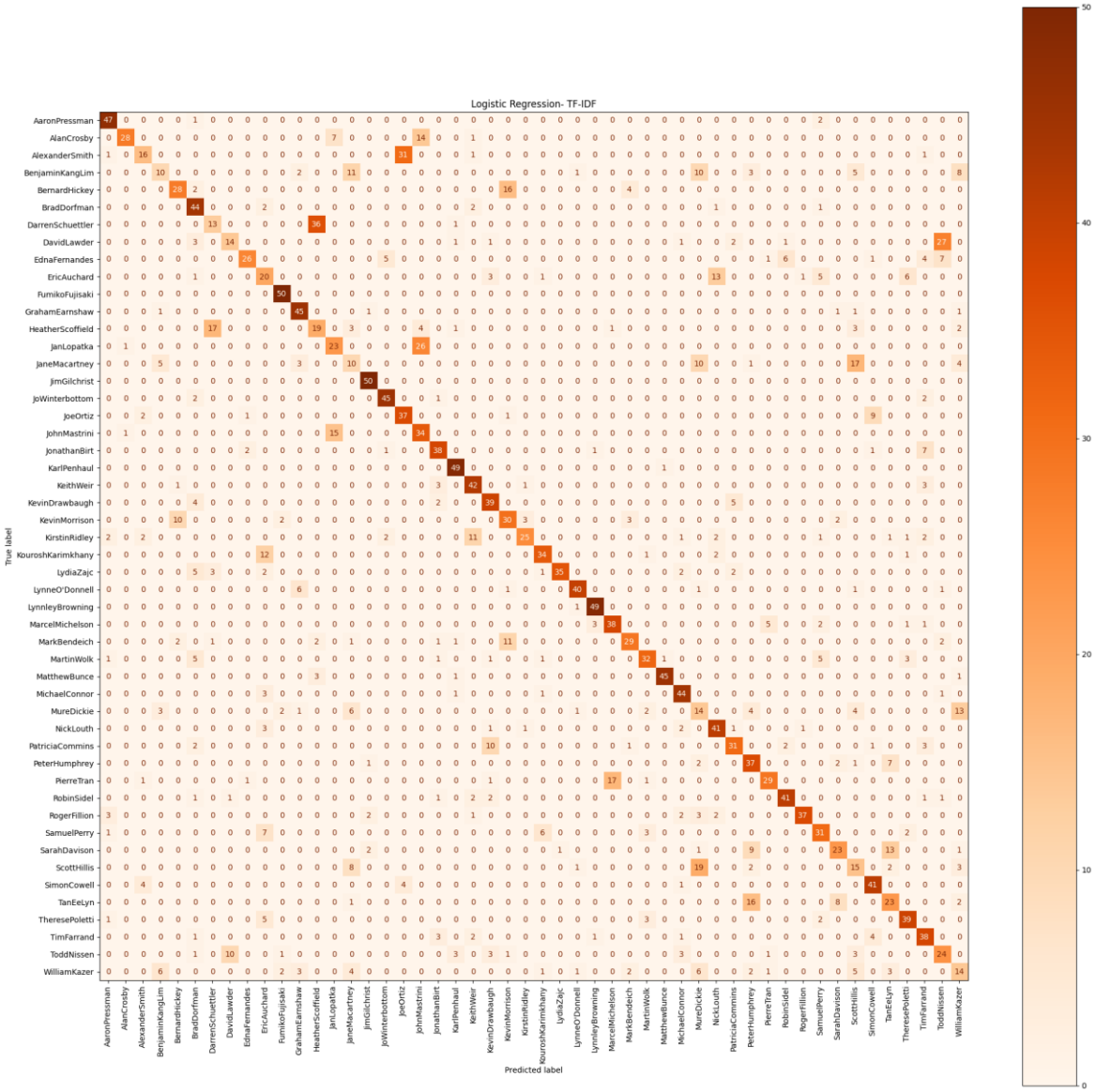
```
Logistic Regression - Bag of words

Accuracy : 0.6424

Precision: 0.6558731505758978

Recall   : 0.6424

F1 Score : 0.6380288292221968
```



Code Description of Logistic Regression (TF-IDF):

This code trains and evaluates a **Logistic Regression** classifier using the **TF-IDF** feature representation. First, a LogisticRegression model is created with max_iter=1000 to ensure that the optimization algorithm has sufficient iterations to converge during training.

Task 11: Support Vector Machine

LinearSVC, a linear Support Vector Machine well suited to high-dimensional sparse text features, is trained and evaluated on both BoW and TF-IDF.

Code (Bag of Words):

```
svm_bow = LinearSVC()

svm_bow.fit(X_train_bow, y_train)

y_pred_svm_bow = svm_bow.predict(X_test_bow)


print("Support Vector Machine - Bag of words")

print("Accuracy :", accuracy_score(y_test, y_pred_svm_bow))

print("Precision:", precision_score(y_test, y_pred_svm_bow,
average="weighted"))

print("Recall   :", recall_score(y_test, y_pred_svm_bow,
average="weighted"))

print("F1 Score :", f1_score(y_test, y_pred_svm_bow, average="weighted"))

of Words")


svm_bow_cm = confusion_matrix(y_test, y_pred_svm_bow)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(

    confusion_matrix=svm_bow_cm,

    display_labels=svm_bow.classes_

).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Support Vector Machine - Bag Of Words")

plt.show()
```

Output **Support Vector Machine** (Bag of words):

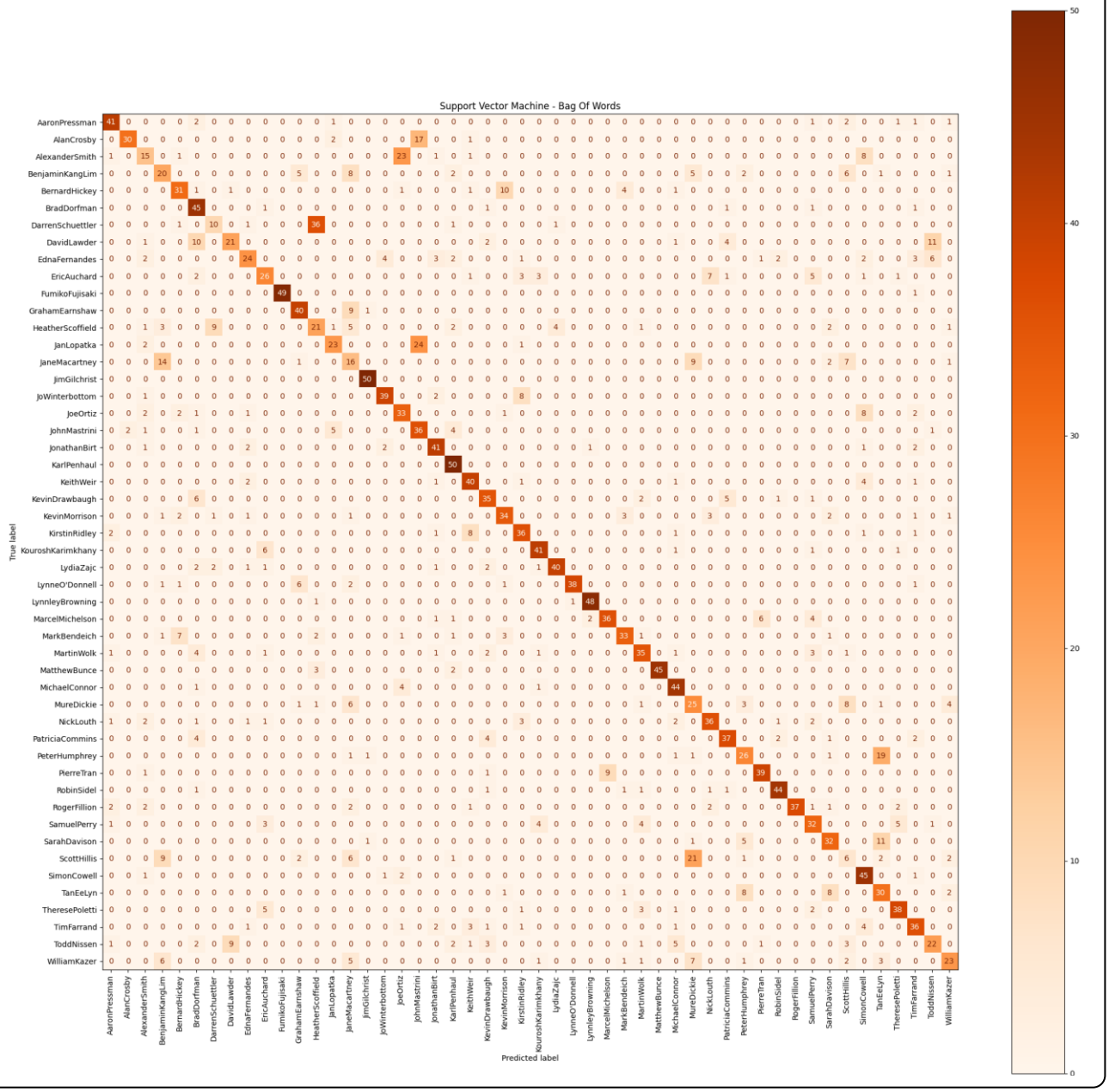
```
Support Vector Machine - Bag of words

Accuracy : 0.6696

Precision: 0.6818951666415747

Recall   : 0.6696

F1 Score : 0.6669761413828533
```



Code Description of Support Vector Machine (BoW):

This code trains and evaluates a **Support Vector Machine (SVM)** classifier using the **Bag of Words (BoW)** feature representation. The LinearSVC classifier is created and trained using the Bag of Words training feature matrix (X_train_bow) and the corresponding author labels

(y_train). After training, the model predicts the author of each document in the testing dataset using the predict() method. The classification performance is then evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-score**. Finally, a **confusion matrix** is generated and visualized using ConfusionMatrixDisplay to illustrate the correct and incorrect predictions for each author class.

Code (TF-IDF):

```
svm_tfidf = LinearSVC()
svm_tfidf.fit(X_train_tfidf, y_train)
y_pred_svm_tfidf = svm_tfidf.predict(X_test_tfidf)


print("Support Vector Machine - TF-IDF")
print("Accuracy :", accuracy_score(y_test, y_pred_svm_tfidf))
print("Precision:", precision_score(y_test, y_pred_svm_tfidf,
average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_svm_tfidf,
average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_svm_tfidf,
average="weighted"))
svm_tfidf_cm = confusion_matrix(y_test, y_pred_svm_tfidf)


fig, ax = plt.subplots(figsize=(25, 25))


ConfusionMatrixDisplay(
    confusion_matrix=svm_tfidf_cm,
    display_labels=svm_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)


plt.title("Support Vector Machine - TF-IDF")
plt.show()
```

Output Logistic Regression (TF-IDF):

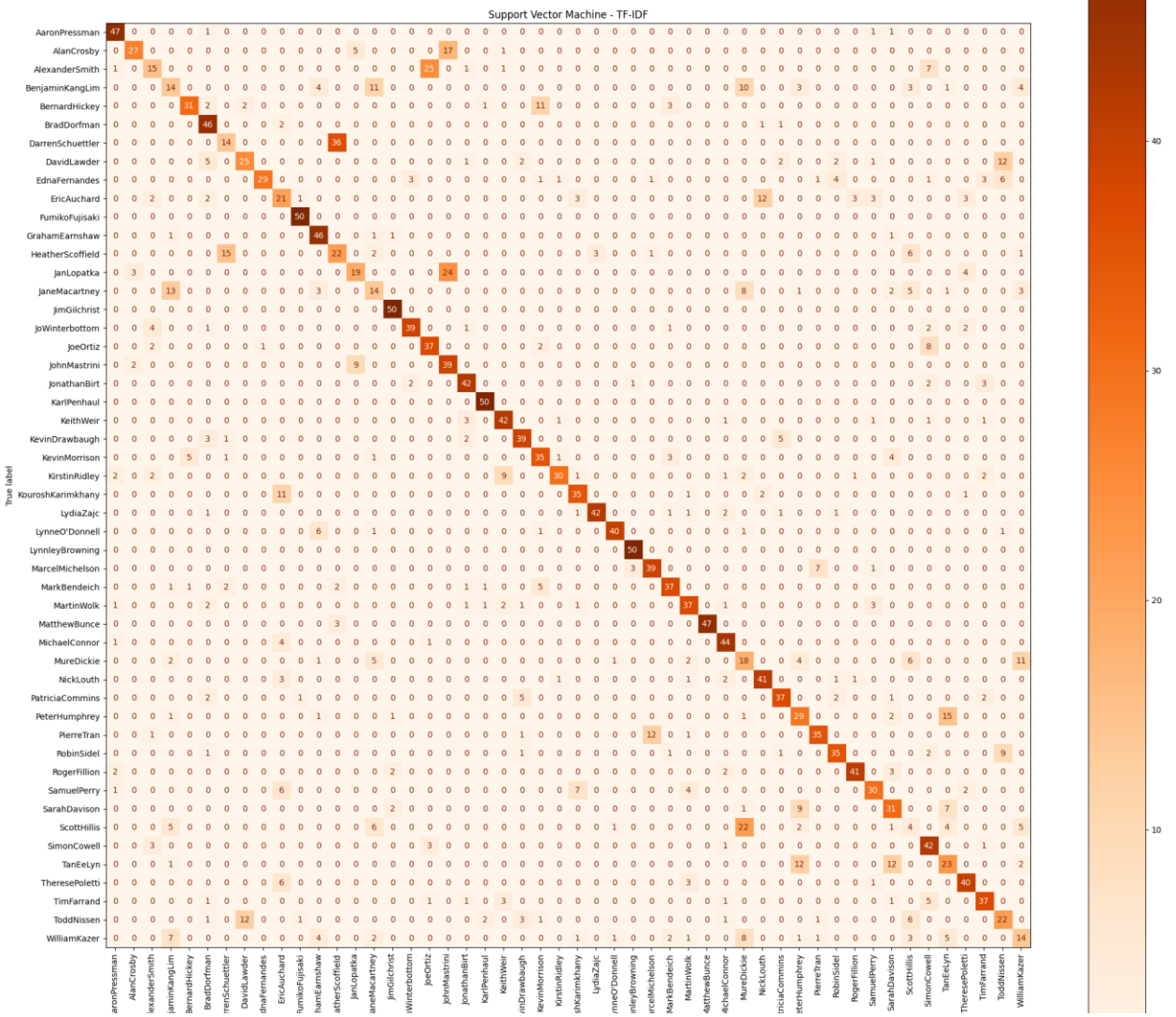
```
Support Vector Machine - Bag of words

Accuracy : 0.6692

Precision: 0.6758593600269829

Recall   : 0.6692

F1 Score : 0.66490950544679
```



Code Description of Logistic Regression (TF-IDF):

This code trains and evaluates a **Support Vector Machine (SVM)** classifier using the **TF-IDF** feature representation. First, a LinearSVC model is created and trained using the TF-IDF training feature matrix (X_train_tfidf) and the corresponding author labels (y_train). After training, the predict() method is used to classify the testing documents and generate the predicted author labels (y_pred_svm_tfidf). The model's performance is evaluated using

Accuracy, **Precision**, **Recall**, and **F1-score** to measure its classification effectiveness. Finally, a **confusion matrix** is generated and displayed using ConfusionMatrixDisplay, allowing visualization of the correctly and incorrectly classified author documents...

Results Comparison

The table below reports test-set accuracy for every Algorithm x Representation x Dataset combination required by the project, using Dataset 1 = Reuters 50-50 (50 authors) and Dataset 2 = Spooky Author Identification (3 authors).

Algorithm	Representation	Dataset 1 (Reuters 50-50)	Dataset 2 (Spooky Author)
Naive Bayes	BoW	Accuracy: 0.6544 Precision: 0.6814 Recall: 0.6544	Accuracy: 0.8368 Precision: 0.8383 Recall: 0.8368
	TF-IDF	Accuracy: 0.632 Precision: 0.6496 Recall: 0.632	Accuracy: 0.8207 Precision: 0.8291 Recall: 0.8207
Logistic Regression	BoW	Accuracy: 0.674 Precision: 0.685 Recall: 0.674	Accuracy: 0.81 Precision: 0.81 Recall: 0.81
	TF-IDF	Accuracy: 0.6424 Precision: 0.6558 Recall: 0.6424	Accuracy: 0.8169 Precision: 0.8199 Recall: 0.8169
Support Vector Machine	BoW	Accuracy: 0.6696 Precision: 0.6818 Recall: 0.6696	Accuracy: 0.7985 Precision: 0.80 Recall: 0.7982
	TF-IDF	Accuracy: 0.6692 Precision: 0.6758 Recall: 0.6692	Accuracy: 0.8192 Precision: 0.8204 Recall: 0.8192

Overall, accuracy is consistently higher on the Spooky Author dataset than on Reuters 50-50, which is expected since Spooky is a 3-class problem while Reuters is a much harder 50-class problem. On Reuters, Logistic Regression with BoW gives the best result (67.4%); on Spooky, Support Vector Machine with TF-IDF gives the best result (83.5%). Across both datasets, TF-IDF does not uniformly beat BoW - the better representation depends on the algorithm, which is why both were required and compared for every model.

Project Code

```
from google.colab import drive
drive.mount('/content/drive')
```

```
REUTERS_TRAIN_DIR = "/content/drive/MyDrive/nlp_midterm/reuters/train.csv"
REUTERS_TEST_DIR =  "/content/drive/MyDrive/nlp_midterm/reuters/test.csv"
Spooky_Aurthor_DIR = "/content/drive/MyDrive/nlp_midterm/Spooky_Author
/Spooky_Author_Dataset.csv"
```

```
import os
import re
import string
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords
from nltk.stem import wordnet
import spacy
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
df1 = pd.read_csv(REUTERS_TRAIN_DIR, encoding ='unicode_escape')
df2 = pd.read_csv(REUTERS_TEST_DIR, encoding ='unicode_escape')


train_data1 = df1[["text", "author"]]
train_data1.dropna(how='any', inplace=True)
print(train_data1.head())

test_data1 = df2[["text", "author"]]
test_data1.dropna(how='any', inplace=True)
print(test_data1.head())
```

```
train_data1["tokens"] = train_data1["text"].apply(word_tokenize)
print(train_data1[["text", "tokens"]].head())


test_data1["tokens"] = test_data1["text"].apply(word_tokenize)
print(test_data1[["text", "tokens"]].head())
```

```
train_data1["tokens"] = [
    [word.lower() for word in words]
    for words in train_data1["tokens"]
]
print(train_data1["tokens"].head())
```

```
test_data1["tokens"] = [
    [word.lower() for word in words]
    for words in test_data1["tokens"]
]
print(test_data1["tokens"].head())
```

```
nlp = spacy.load("en_core_web_sm")
```

```
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)

print(train_data1["tokens"].head())
```

```
test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)

print(test_data1["tokens"].head())
```

```
stop_words = set(stopwords.words("english"))
```

```
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)

print(train_data1["tokens"].head())
```

```
test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)

print(test_data1["tokens"].head())
```

```
train_data1["tokens"] = train_data1["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)

print(train_data1["tokens"].head())

test_data1["tokens"] = test_data1["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)

print(test_data1["tokens"].head())
```

```
train_data1["processed_text"] = train_data1["tokens"].apply(" ".join)

train_data1.head()
```

```
test_data1["processed_text"] = test_data1["tokens"].apply(" ".join)

test_data1.head()
```

```
bow = CountVectorizer()

X_train_bow = bow.fit_transform(train_data1["processed_text"])
X_test_bow = bow.transform(test_data1["processed_text"])

y_train = train_data1["author"]
y_test = test_data1["author"]

print(X_train_bow.shape)
print(X_test_bow.shape)
```

```
tfidf = TfidfVectorizer()

X_train_tfidf = tfidf.fit_transform(train_data1["processed_text"])
X_test_tfidf = tfidf.transform(test_data1["processed_text"])
print(X_train_tfidf.shape)
print(X_test_tfidf.shape)
```

```
mnb_bow = MultinomialNB()

mnb_bow.fit(X_train_bow, y_train)

y_pred_mnb_bow = mnb_bow.predict(X_test_bow)
```

```
print("Naive Bayes-Bag of words")
print("Accuracy :", accuracy_score(y_test, y_pred_mnb_bow))
print("Precision:", precision_score(y_test, y_pred_mnb_bow, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_mnb_bow, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_mnb_bow, average="weighted"))
```

```
mnb_bow_cm = confusion_matrix(y_test, y_pred_mnb_bow)
fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=mnb_bow_cm,
    display_labels=mnb_bow.classes_
).plot(ax=ax, cmap="Greens",xticks_rotation=90)

plt.title("Multinomial Naive Bayes - Bag of Words")
plt.show()
```

```
mnb_tfidf = MultinomialNB()

mnb_tfidf.fit(X_train_tfidf, y_train)

y_pred_mnb_tfidf = mnb_tfidf.predict(X_test_tfidf)
```

```
print("Naive Bayes - TF-IDF")
print("Accuracy :", accuracy_score(y_test, y_pred_mnb_tfidf))
print("Precision:", precision_score(y_test, y_pred_mnb_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_mnb_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_mnb_tfidf, average="weighted"))
```

```
mnb_tfidf_cm = confusion_matrix(y_test, y_pred_mnb_tfidf)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=mnb_tfidf_cm,
    display_labels=mnb_tfidf.classes_
).plot(ax=ax, cmap="Blues", xticks_rotation=90)

plt.title("Multinomial Naive Bayes - TF-IDF")
plt.show()
```

```
lr_bow = LogisticRegression(max_iter=1000)

lr_bow.fit(X_train_bow, y_train)

y_pred_lr_bow = lr_bow.predict(X_test_bow)
```

```
print("Logistic Regression - Bag of words")
print("Accuracy :", accuracy_score(y_test, y_pred_lr_bow))
print("Precision:", precision_score(y_test, y_pred_lr_bow, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_bow, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_bow, average="weighted"))
```

```
lr_bow_cm = confusion_matrix(y_test, y_pred_lr_bow)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=lr_bow_cm,
    display_labels=lr_bow.classes_
).plot(ax=ax, cmap="Reds", xticks_rotation=90)

plt.title("Logistic Regression-BoW")
plt.show()
```

```
lr_tfidf = LogisticRegression(max_iter=1000)

lr_tfidf.fit(X_train_tfidf, y_train)

y_pred_lr_tfidf = lr_tfidf.predict(X_test_tfidf)
```

```
print("Logistic Regression - TF-IDF")
print("Accuracy :", accuracy_score(y_test, y_pred_lr_tfidf))
print("Precision:", precision_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_lr_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_lr_tfidf, average="weighted"))
```

```
lr_tfidf_cm = confusion_matrix(y_test, y_pred_lr_tfidf)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=lr_tfidf_cm,
    display_labels=lr_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Logistic Regression- TF-IDF")
plt.show()
```

```
svm_bow = LinearSVC()
```

```
svm_bow.fit(X_train_bow, y_train)

y_pred_svm_bow = svm_bow.predict(X_test_bow)
```

```
print("Support Vector Machine - Bag of words")
print("Accuracy :", accuracy_score(y_test, y_pred_svm_bow))
print("Precision:", precision_score(y_test, y_pred_svm_bow, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_svm_bow, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_svm_bow, average="weighted"))
```

```
svm_bow_cm = confusion_matrix(y_test, y_pred_svm_bow)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=svm_bow_cm,
    display_labels=svm_bow.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Support Vector Machine - Bag Of Words")
plt.show()
```

```
svm_tfidf = LinearSVC()

svm_tfidf.fit(X_train_tfidf, y_train)

y_pred_svm_tfidf = svm_tfidf.predict(X_test_tfidf)
```

```
print("Support Vector Machine - TF-IDF")
print("Accuracy :", accuracy_score(y_test, y_pred_svm_tfidf))
print("Precision:", precision_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test, y_pred_svm_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test, y_pred_svm_tfidf, average="weighted"))
```

```
svm_tfidf_cm = confusion_matrix(y_test, y_pred_svm_tfidf)

fig, ax = plt.subplots(figsize=(25, 25))

ConfusionMatrixDisplay(
    confusion_matrix=svm_tfidf_cm,
    display_labels=svm_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Support Vector Machine - TF-IDF")
plt.show()
```

```
df3 = pd.read_csv(Spooky_Aurthor_DIR)

train_data2, test_data2 = train_test_split(
    df3,
    test_size=0.2,
    random_state=42,
    stratify=df3["author"]
)

train_data2.dropna(how='any', inplace=True)
print(train_data2.head())

test_data2.dropna(how='any', inplace=True)
print(test_data2.head())
```

```
train_data2["tokens"] = train_data2["text"].apply(word_tokenize)
print(train_data2[["text", "tokens"]].head())


test_data2["tokens"] = test_data2["text"].apply(word_tokenize)
print(test_data2[["text", "tokens"]].head())
```

```
train_data2["tokens"] = [
    [word_tokenized.lower() for word_tokenized in words]
    for words in train_data2["tokens"]
]
print(train_data2["tokens"].head())


test_data2["tokens"] = [
    [word_tokenized.lower() for word_tokenized in words]
    for words in test_data2["tokens"]
]
print(test_data2["tokens"].head())
```

```
train_data2["tokens"] = train_data2["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)

print(train_data2["tokens"].head())

test_data2["tokens"] = test_data2["tokens"].apply(
    lambda words: [token.lemma_ for token in nlp(" ".join(words))]
)

print(test_data2["tokens"].head())
```

```
train_data2["tokens"] = train_data2["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)

print(train_data2["tokens"].head())




test_data2["tokens"] = test_data2["tokens"].apply(
    lambda words: [word for word in words if word not in stop_words]
)

print(test_data1["tokens"].head())
```

```
train_data2["tokens"] = train_data2["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)

print(train_data2["tokens"].head())

test_data2["tokens"] = test_data2["tokens"].apply(
    lambda words: [word for word in words if word not in string.punctuation]
)

print(test_data2["tokens"].head())
```

```
train_data2["processed_text"] = train_data2["tokens"].apply(" ".join)
```

```
train_data2.head()
```

```
test_data2["processed_text"] = test_data2["tokens"].apply(" ".join)

test_data2.head()
```

```
bow = CountVectorizer()

X_train2_bow = bow.fit_transform(train_data2["processed_text"])
X_test2_bow = bow.transform(test_data2["processed_text"])

y_train2 = train_data2["author"]
y_test2 = test_data2["author"]

print(X_train2_bow.shape)
print(X_test2_bow.shape)
```

```
tfidf = TfidfVectorizer()

X_train2_tfidf = tfidf.fit_transform(train_data2["processed_text"])
X_test2_tfidf = tfidf.transform(test_data2["processed_text"])
print(X_train2_tfidf.shape)
print(X_test2_tfidf.shape)
```

```
mnb_bow = MultinomialNB()

mnb_bow.fit(X_train2_bow, y_train2)

y_pred2_mnb_bow = mnb_bow.predict(X_test2_bow)
```

```
print("Naive Bayes - Bag Of Words")
print("Accuracy :", accuracy_score(y_test2, y_pred2_mnb_bow))
print("Precision:", precision_score(y_test2, y_pred2_mnb_bow, average="weighted"))
print("Recall   :", recall_score(y_test2, y_pred2_mnb_bow, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_mnb_bow, average="weighted"))
```

```
mnb_bow_cm2 = confusion_matrix(y_test2, y_pred2_mnb_bow)
fig, ax = plt.subplots(figsize=(5, 5))

ConfusionMatrixDisplay(
    confusion_matrix=mnb_bow_cm2,
    display_labels=mnb_bow.classes_
).plot(ax=ax, cmap="Greens", xticks_rotation=90)

plt.title("Multinomial Naive Bayes - Bag of Words")
plt.show()
```

```
mnb_tfidf = MultinomialNB()

mnb_tfidf.fit(X_train2_tfidf, y_train2)

y_pred2_mnb_tfidf = mnb_tfidf.predict(X_test2_tfidf)
```

```
print("Naive Bays - TF-IDF")
print("Accuracy :", accuracy_score(y_test2, y_pred2_mnb_tfidf))
print("Precision:", precision_score(y_test2, y_pred2_mnb_tfidf, average="weighted"))
```

```
print("Recall   :", recall_score(y_test2, y_pred2_mnb_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_mnb_tfidf, average="weighted"))
```

```
mnb_tfidf_cm = confusion_matrix(y_test2, y_pred2_mnb_tfidf)

fig, ax = plt.subplots(figsize=(5, 5))

ConfusionMatrixDisplay(
    confusion_matrix=mnb_tfidf_cm,
    display_labels=mnb_tfidf.classes_
).plot(ax=ax, cmap="Blues", xticks_rotation=90)

plt.title("Multinomial Naive Bayes - TF-IDF")
plt.show()
```

```
lr_bow = LogisticRegression(max_iter=1000)

lr_bow.fit(X_train2_bow, y_train2)

y_pred2_lr_bow = lr_bow.predict(X_test2_bow)
```

```
print(Logistic Regression - TF-IDF)
print("Accuracy :", accuracy_score(y_test2, y_pred2_lr_bow))
print("Precision:", precision_score(y_test2, y_pred2_lr_bow, average="weighted"))
print("Recall   :", recall_score(y_test2, y_pred2_lr_bow, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_lr_bow, average="weighted"))
```

```
lr_bow_cm2 = confusion_matrix(y_test2, y_pred2_lr_bow)

fig, ax = plt.subplots(figsize=(5,5))

ConfusionMatrixDisplay(
    confusion_matrix=lr_bow_cm2,
    display_labels=lr_bow.classes_
).plot(ax=ax, cmap="Reds", xticks_rotation=90)

plt.title("Logistic Regression-BoW")
plt.show()
```

```
lr_tfidf = LogisticRegression(max_iter=1000)

lr_tfidf.fit(X_train2_tfidf, y_train2)

y_pred2_lr_tfidf = lr_tfidf.predict(X_test2_tfidf)
```

```
print("Logistic Regression - TF-IDF")
print("Accuracy :", accuracy_score(y_test2, y_pred2_lr_tfidf))
print("Precision:", precision_score(y_test2, y_pred2_lr_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test2, y_pred2_lr_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_lr_tfidf, average="weighted"))
```

```
lr_tfidf_cm2 = confusion_matrix(y_test2, y_pred2_lr_tfidf)

fig, ax = plt.subplots(figsize=(5, 5))

ConfusionMatrixDisplay(
    confusion_matrix=lr_tfidf_cm2,
    display_labels=lr_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Logistic Regression- TF-IDF")
```

```
plt.show()
```

```
svm_bow = LinearSVC()

svm_bow.fit(X_train2_bow, y_train2)

y_pred2_svm_bow = svm_bow.predict(X_test2_bow)
```

```
print("Support Vector Machine - Bag of Word")
print("Accuracy :", accuracy_score(y_test2, y_pred2_svm_bow))
print("Precision:", precision_score(y_test2, y_pred2_svm_bow, average="weighted"))
print("Recall   :", recall_score(y_test2, y_pred2_svm_bow, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_svm_bow, average="weighted"))
```

```
svm_bow_cm2 = confusion_matrix(y_test2, y_pred2_svm_bow)

fig, ax = plt.subplots(figsize=(5, 5))

ConfusionMatrixDisplay(
    confusion_matrix=svm_bow_cm2,
    display_labels=lr_tfidf.classes_
).plot(ax=ax, cmap="Oranges", xticks_rotation=90)

plt.title("Support Vector Machine - Bag Of Words")
plt.show()
```

```
svm_tfidf = LinearSVC()

svm_tfidf.fit(X_train2_tfidf, y_train2)

y_pred2_svm_tfidf = svm_tfidf.predict(X_test2_tfidf)
```

```
print("Accuracy :", accuracy_score(y_test2, y_pred2_svm_tfidf))
print("Precision:", precision_score(y_test2, y_pred2_svm_tfidf, average="weighted"))
print("Recall   :", recall_score(y_test2, y_pred2_svm_tfidf, average="weighted"))
print("F1 Score :", f1_score(y_test2, y_pred2_svm_tfidf, average="weighted"))
```

```
print(classification_report(y_test2, y_pred2_svm_tfidf))
```

```
lr_tfidf_cm2 = confusion_matrix(y_test2, y_pred2_svm_tfidf)

fig, ax = plt.subplots(figsize=(5, 5))

ConfusionMatrixDisplay(
    confusion_matrix=lr_tfidf_cm2,
    display_labels=lr_tfidf.classes_
).plot(ax=ax, cmap="Blues", xticks_rotation=90)

plt.title("Logistic Regression- TF-IDF")
plt.show()
```